Eventos

Objetivo:

- Conocer los conceptos relacionados con eventos
- Desarrollar aplicaciones con clases publicadoras que generan eventos.

Un evento es el mecanismo mediante el cual una clase puede proporcionar notificaciones a sus clientes cuando ocurre algún suceso importante con sus objetos, mismo que es generado por una clase publicadora y notificado a un conjunto de objetos de clases clientes o suscriptoras.

Esto quiere decir que si un objeto expone un evento, entonces cualquier objeto de cualquier clase puede suscribirse a dicho evento.

La implementación de eventos más común se presenta en aplicaciones visuales, donde las clases de los controles de la interfaz gráfica disponen de eventos que se notifican cuando el usuario realiza alguna actividad con el control (como hacer click en algún botón de la forma).

Sin embargo, es importante mencionar que los eventos no son exclusivos para las aplicaciones visuales, ya que una aplicación de consola puede tener clases que disponen de eventos.

Los eventos son una forma por la cual una clase notifica a otras clases u objetos que sucedió algo de interés.

La clase que envía la notificación se conoce como editor. La clase que recibe la notificación se conoce como suscriptor del evento.

El uso de un evento se compone de dos partes:

1. La clase cuyos objetos generan el evento (publicadora).
2. Las clases cuyos objetos reciben el evento (suscriptoras).

Los eventos se declaran a través de delegados. Un evento es la manera en que una clase le permite a sus clientes proporcionar delegados a los métodos para invocarlos al momento en que se genera el evento. Esto significa que cuando se produce el evento, entonces se llama a los delegados que proporcionan los clientes para dicho evento.

En pocas palabras, se utiliza un evento cuando desde una clase publicadora se desea informar sobre algún suceso a otras clases cliente

Para declarar un evento en una clase, primero debe definirse un tipo delegado para dicho evento.

Al declarar el delegado se utiliza el postfijo EventHandler para identificarlo y asumir que se trata de la definición de un delegado para un evento.

Por ejemplo, si al modificar el valor de un atributo o propiedad se desea generar un evento y enviar un mensaje correspondiente, entonces se declara un delegado llamado ModificarEventHandler()

```
//delegado
piblic delegate void ModificarEventHandler(string strMensaje);
```

La clase que genera el evento utiliza la declaración del delegado para definirlo.

Siguiendo con la declaración del delegado anterior, se define un evento llamado Modificacion con la definición del delegado ModificarEventHandler

```
// Evento
public event ModificarEventHandler Modificacion;
```

Una vez declarado el delegado y habiéndolo utilizado para definir el evento, la clase está preparada para lanzar notificaciones, generando eventos a través de su nombre con los parámetros definidos en el delegado.

Por ejemplo, si una propiedad de la clase publicadora desea lanzar una notificación cuando se modifica el valor de su atributo, entonces genera un evento con un mensaje.

```
// Delegado definido por el programador
public delegate void ModificarEventHandler(string strMensaje);
// Definición de la clase que genera el evento (clase publicadora)
Public class ClasePublicadora
{
        // Atributos, métodos y propiedades
        . . . . .
        // Evento
        public event ModificarEventHandler Modificacion;
        // Propiedad que genera el evento
        public void Propiedad
        {
                set
                {
                        // Modifica el valor de su atributo
                        . . . . .
                        // Genera el evento y dispara una notificación
                        this.Modificacion("Mensaje generado por el evento");
                }
        }
}
```

¿Cómo suscribirse a un evento?

Cuando se utilizan delegados y eventos, se manejan los conceptos de la clase publicadora y la clase suscriptora.

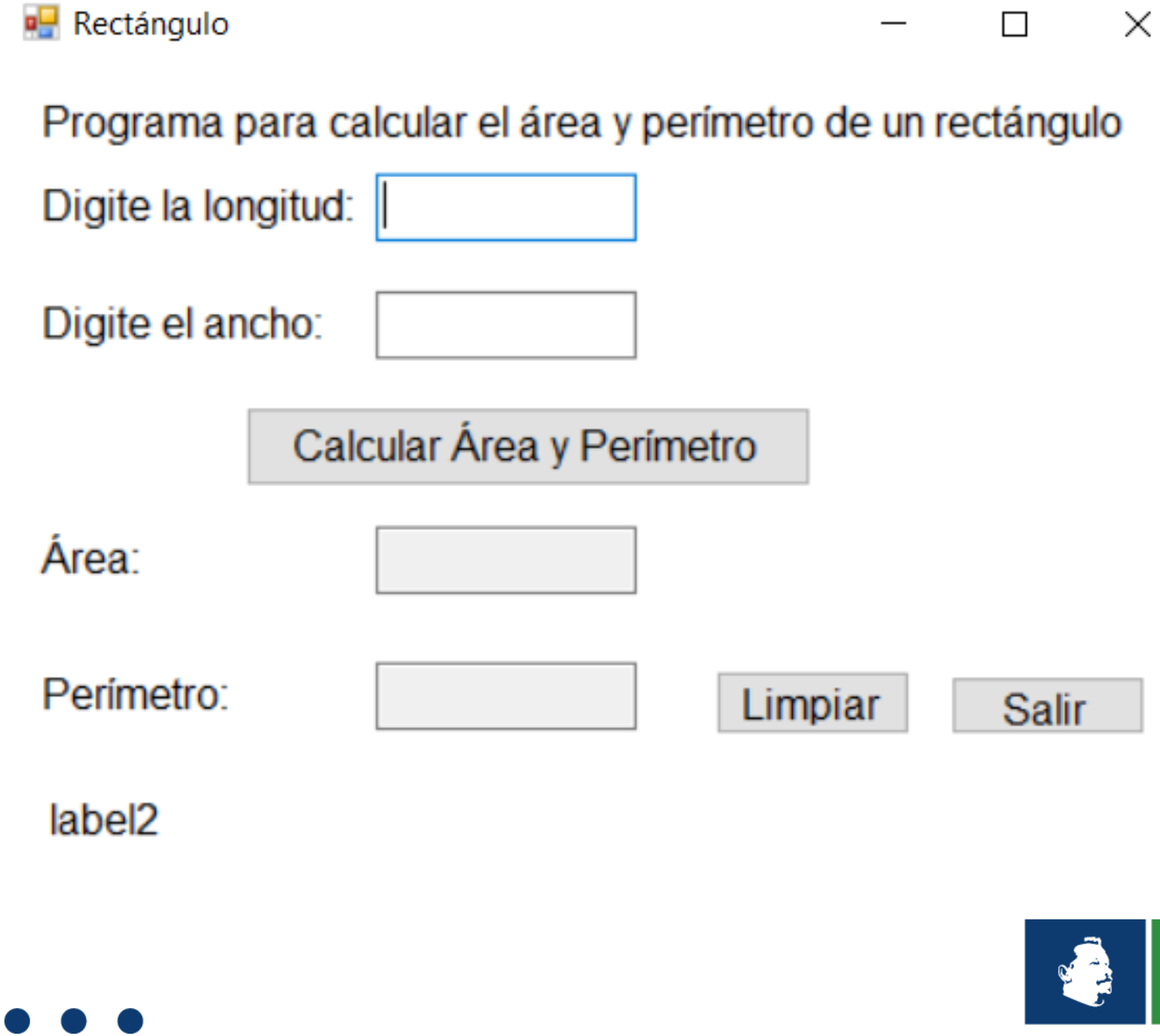
La clase publicadora es la que genera, lanza o dispara el evento, mientras La clase suscriptora es la interesada en recibir las notificaciones de la clase publicadora.

La clase suscriptora utiliza el operador += para prepararse y recibir notificaciones de los eventos generados en la clase publicadora.

```
// Suscripción al evento
miObjeto.Modificacion+= new ModificarEventHandler(MetodoInvocadoConEvento);
```

Es más sencilla de suscribirse a un evento; basta con utilizar el operador **+=** y después escribir el nombre del método gestor (sin necesidad de especificar el delegado)

// Otra forma de suscripción al evento
miObjeto.Modificacion += MetodoInvocadoConEvento;


Rectángulo
Programa para calcular el área y perímetro de un rectángulo
Digite la longitud:
Digite el ancho:
Calcular Área y Perímetro
Área:
Perímetro:
Limpiar
Salir
label2

Ejemplo 1

```
6 referencias
public class Rectangulo//clase editor
{
    //atributos
    private double longitud;
    private double ancho;
    //El delegado del evento LongitudChanged es de tipo EventHandler, retorno void, acepta dos parametros object sender, EventArgs e
    //El evento EventHandler no retorna datos
    public event EventHandler LongitudChanged;

    //constructor por defecto
    1 referencia
    public Rectangulo() { }

    //constructor: como inicializar un objeto de clase Rectangulo
    0 referencias
    public Rectangulo(double longitud, double ancho)
    {
        this.longitud = longitud;
        this.ancho = ancho;
    }
```

Ejemplo 1

```
    //propiedades
    4 referencias
    public double Longitud
    {
        get { return longitud; }
        set
        {
            longitud = value;
            //evento LongitudChanged se invoca cuando se llama accessor set de la propiedad Longitud
            LongitudChanged(this, EventArgs.Empty);
            //this apunta al objeto que es la instancia de la clase Rectangulo
            //object sender el que genera el evento
            //EventArgs los datos del evento, los datos que queremos pasar al manejador de eventos
        }
    }
    3 referencias
    public double Ancho
    {
        get { return ancho; }
        set { ancho = value; }
    }

    //métodos
    1 referencia
    public double GetArea()
    {
        return Longitud * Ancho;
    }
    1 referencia
    public double GetPerimetro()
    {
        return (2 * Longitud) + (2 * Ancho);
    }
}
```

UFG
UNIVERSIDAD FRANCISCO GAVIDIA
Powered by Arizona State University®

Ejemplo 1

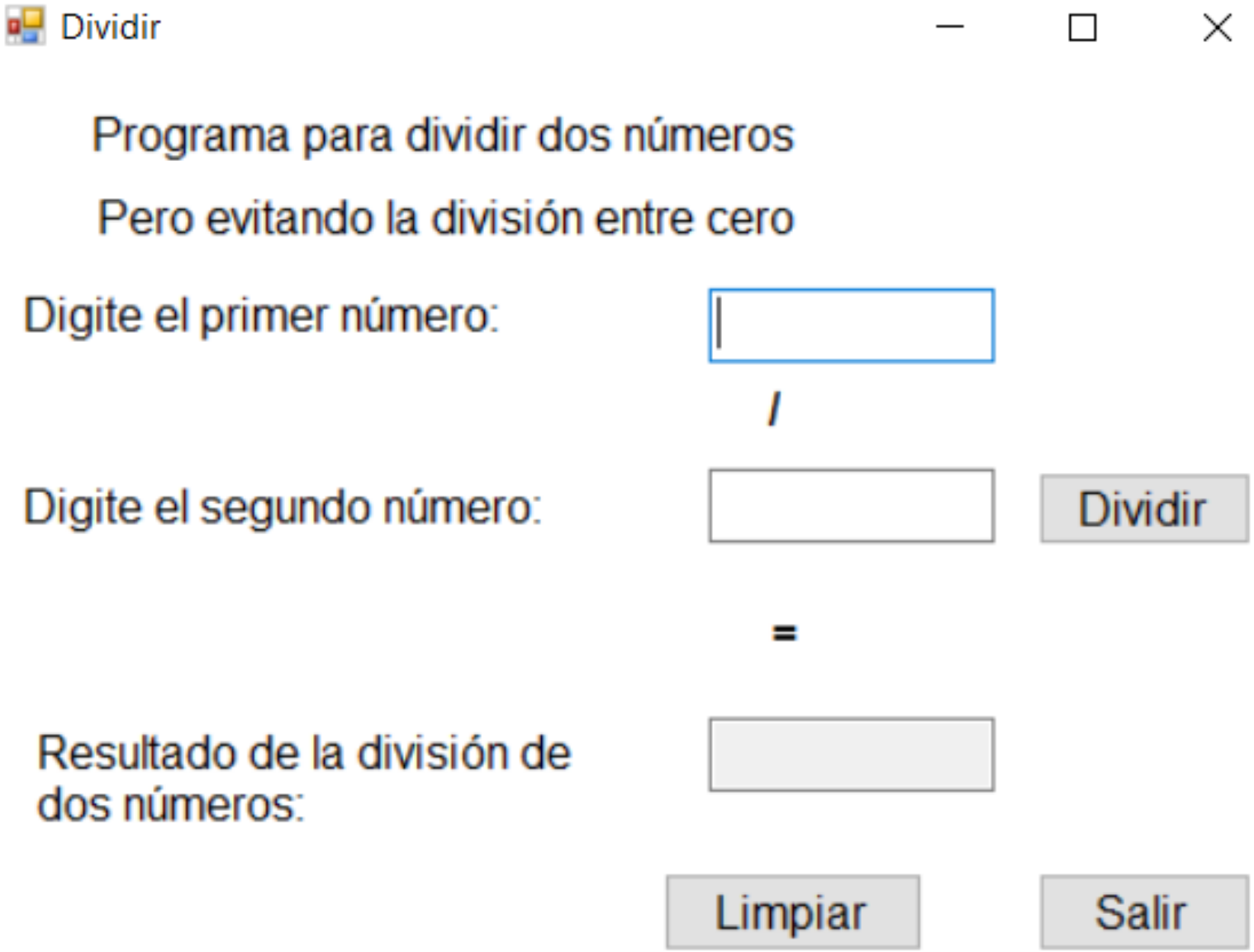
```
1 referencia
private void btnCalcular_Click(object sender, EventArgs e)
{
    Rectangulo rectangulo = new Rectangulo();
    //suscribirse al evento LongitudChanged
    //rectangulo.LongitudChanged += Rectangulo_LongitudChanged;
    rectangulo.LongitudChanged += new EventHandler(Rectangulo_LongitudChanged);
    rectangulo.Longitud = Convert.ToDouble(txtLongitud.Text);
    rectangulo.Ancho = Convert.ToDouble(txtAncho.Text);

    txtArea.Text = rectangulo.GetArea().ToString();
    txtPerimetro.Text = rectangulo.GetPerimetro().ToString();
}
//El manejador del evento
//La firma del método Rectangulo_LongitudChanged coincide con requerimientos del delegado EventHandler
//El método Rectangulo_LongitudChanged se invoca tan pronto se establece el valor de la propiedad  Longitud
1 referencia
private void Rectangulo_LongitudChanged(object sender, EventArgs e)
{
    Rectangulo rect = (Rectangulo)sender;
    lblNotificarCambio.Text = "Valor cambiado: longitud = " + rect.Longitud;
}

1 referencia
private void btnLimpiar_Click(object sender, EventArgs e)
{
    txtLongitud.Clear(); txtAncho.Clear(); txtArea.Clear(); txtPerimetro.Clear();
}

1 referencia
private void btnSalir_Click(object sender, EventArgs e)
{
    this.Close();
}
```

UFG
UNIVERSIDAD FRANCISCO GAVIDIA
Powered by Arizona State University®


Dividir
Programa para dividir dos números
Pero evitando la división entre cero
Digite el primer número:
/
Digite el segundo número:
Dividir
=
Resultado de la división de dos números:
Limpiar
Salir

Ejemplo 2

```
3 referencias
public class Operaciones//editora
{
    //Si queremos devolver datos necesitamos nuestro propio delegado
    public delegate void DividirPorCeroEventHandler(object sender, EventArgs e);
    public event DividirPorCeroEventHandler DividirPorCero;

    1 referencia
    public Operaciones() { }

    1 referencia
    public double Dividir(double num1, double num2)
    {
        double resultado = 0;
        if (num2 == 0)
        {
            if (DividirPorCero != null)
            {
                //pide dos parametros el evento
                DividirPorCero(this, EventArgs.Empty);
            }
        }
        else
        {
            resultado = num1 / num2;
        }
        return resultado;
    }
}
```

Ejemplo 2

```
private void btnDividir_Click(object sender, EventArgs e)
{
    double resultado = 0;
    Operaciones operaciones = new Operaciones();
    //suscripcion
    operaciones.DividirPorCero += Operaciones_DividirPorCero;

    resultado = operaciones.Dividir(Convert.ToDouble(txtPrimerNumero.Text), Convert.ToDouble(txtSegundoNumero.Text));
    txtResultado.Text = resultado.ToString();
}

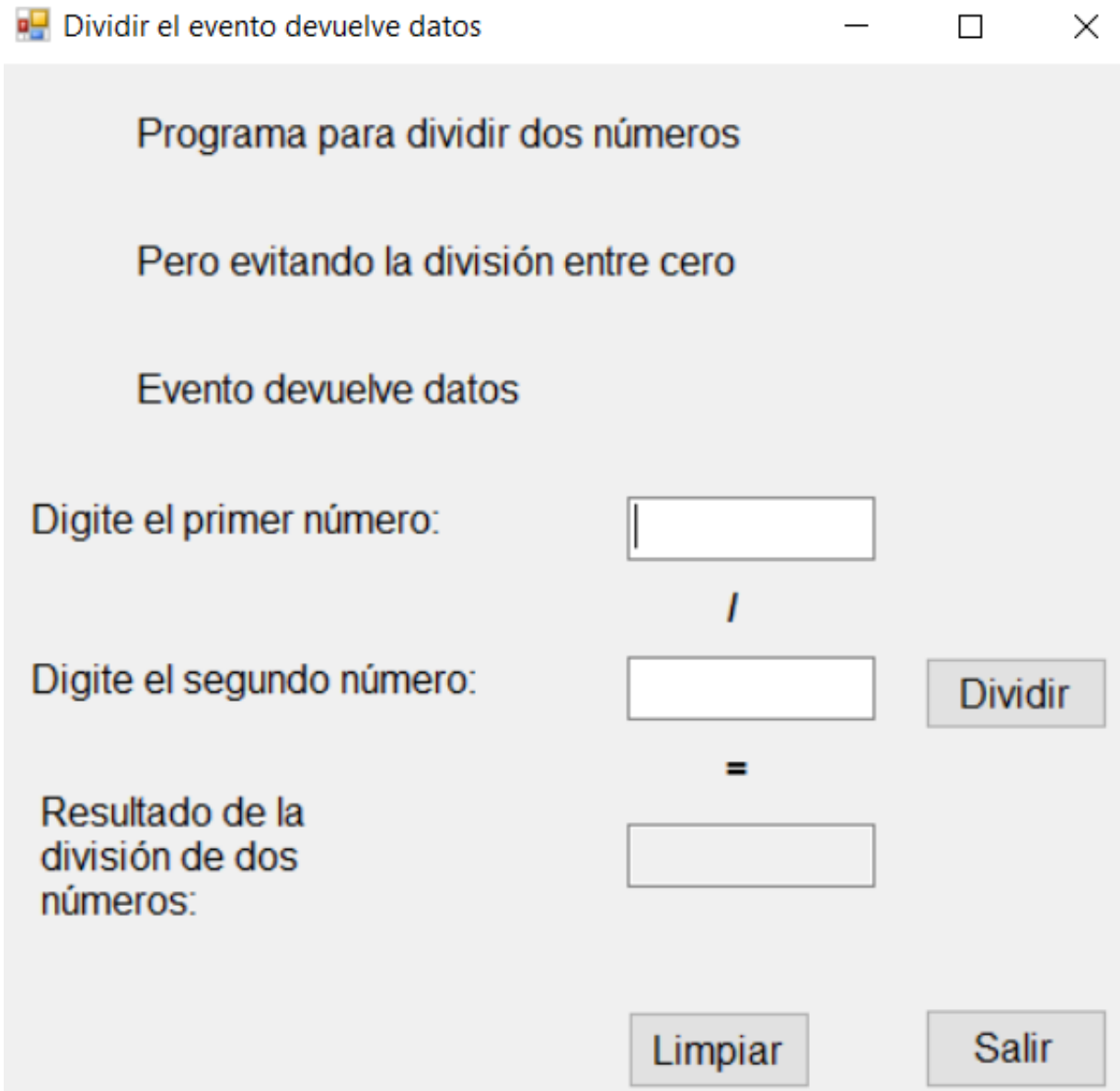
//manejador de eventos
1 referencia
private void Operaciones_DividirPorCero(object sender, EventArgs e)
{
    MessageBox.Show("No es posible dividir entre cero");
}

1 referencia
private void btnLimpiar_Click(object sender, EventArgs e)
{
    txtPrimerNumero.Clear();
    txtSegundoNumero.Clear();
    txtResultado.Clear();
}

1 referencia
private void btnSalir_Click(object sender, EventArgs e)
{
    this.Close();
}
```

UFG
VERSIDAD FRANCISCO GAVIDIA
Powered by Arizona State University®

Ejemplo 3


Dividir el evento devuelve datos
Programa para dividir dos números
Pero evitando la división entre cero
Evento devuelve datos
Digite el primer número:
/
Digite el segundo número:
Dividir
=
Resultado de la división de dos números:
Limpiar
Salir

```
5 referencias
public class DividirPorCeroEventArgs : EventArgs
{
    private double num1;
    private double num2;

    1 referencia
    public DividirPorCeroEventArgs() { }
    0 referencias
    public DividirPorCeroEventArgs(double num1, double num2)
    {
        this.Num1 = num1;
        this.Num2 = num2;
    }

    3 referencias
    public double Num1 { get { return num1; } set { num1 = value; } }
    3 referencias
    public double Num2 { get { return num2; } set { num2 = value; } }
}
```

Ejemplo 3

```
//Si queremos devolver datos necesitamos nuestro propio delegado
//utilizando la clase DividirPorCeroEventArgs
public delegate void DividirPorCeroEventHandler(object sender, DividirPorCeroEventArgs e);
public event DividirPorCeroEventHandler DividirPorCero;
1 referencia
public Operaciones() { }

1 referencia
public double Dividir(double num1, double num2)
{
    double resultado = 0;
    if (num2 == 0)
    {
        if (DividirPorCero != null)
        {
            //DividirPorCeroEventArgs d = new DividirPorCeroEventArgs();
            //d.Num1 = num1; d.Num2 = num2;
            DividirPorCero(this, new DividirPorCeroEventArgs() { Num1 = num1, Num2 = num2 });
            //DividirPorCero(this, d);
        }
    }
    else
    {
        resultado = num1 / num2;
    }
    return resultado;
}
```

```
1 referencia
private void btnDividir_Click(object sender, EventArgs e)
{
    double resultado = 0;
    Operaciones operaciones = new Operaciones();
    //suscribir al evento de la clase editora
    operaciones.DividirPorCero += Operaciones_DividirPorCero;
    resultado = operaciones.Dividir(Convert.ToDouble(txtPrimerNumero.Text), Convert.ToDouble(txtSegundoNumero.Text));
    txtResultado.Text = resultado.ToString();
}

//manejador de evento,
1 referencia
private void Operaciones_DividirPorCero(object sender, DividirPorCeroEventArgs e)
{
    MessageBox.Show($"No es posible dividir entre cero, valores pasados primer numero: {e.Num1}, sgundo numero {e.Num2}");
}

1 referencia
private void btnLimpiar_Click(object sender, EventArgs e)
{
    txtPrimerNumero.Clear(); txtSegundoNumero.Clear(); txtResultado.Clear();
}

1 referencia
private void btnSalir_Click(object sender, EventArgs e)
{
    this.Close();
}
```

- Utilizamos la palabra clave event junto con delegados para declarar eventos.
- Debemos comprobar si un evento es nulo o no antes de ejecutarlo.
- El editor determina cuando un evento es lanzado. el suscriptor determina qué acción o método ejecutar cuando ese evento sucede.
- Para suscribirse a un evento debemos utilizar el operador +=
- Para ejecutar un evento debemos invocar el delegado.

Gracias

Ingeniería y Sistemas

UFG
UNIVERSIDAD FRANCISCO GAVIDIA
Powered by Arizona State University®